

Víctor Prim Romero

Desarrollador backend · IA aplicada

victorprim.com · github.com/PRIMCODIN · vprimromero@gmail.com · Viladecans, Barcelona

Estudiante de 2.º de DAM orientado a backend e IA aplicada. Construyo software de extremo a extremo: APIs en Python, sistemas RAG, agentes con herramientas y despliegue en Linux con Docker. Mi proyecto principal es un chatbot RAG multi-tenant en producción que responde preguntas sobre mí en victorprim.com, con evaluación automática, control de costes y medición de latencia. También he orquestado IA con n8n, Dify y Make en unas prácticas en equipo y en proyectos propios.

PROYECTOS Y PRÁCTICAS DESTACADOS

Chatbot RAG multi-tenant: asistente del portfolio (proyecto personal, en producción)

2026 · [Demo en victorprim.com](#) · [Caso de estudio](#)

- API en FastAPI con streaming SSE y un agente con herramientas para capturar contactos y derivar a una persona. Es multi-tenant, con el aislamiento entre clientes verificado por tests.
- Retrieval sobre PostgreSQL con pgvector (índice HNSW) y embeddings bge-m3 servidos en local con Ollama. Cambié de modelo de embeddings tras medir que el anterior no distinguía bien en español, y el eval pasó de 11/20 a 17/20. Fusioné dos consultas por posición para resolver las preguntas cortas, que pasaron de 10/12 a 12/12.
- Evaluación automática por tenant: eval de retrieval (19/20) y eval conversacional con regresiones permanentes sacadas de fallos reales. Devuelve código de error si algo falla.
- Control de costes: presupuesto diario de tokens por tenant, tope por IP, rate limiting detrás de proxy y límites de turnos y de tokens de salida.
- Medición de latencia por etapa, guardada en base de datos. Detectó que recargar el modelo de embeddings añadía unos 5 s al primer token.
- Desplegado en un VPS Linux con Docker Compose y Caddy (HTTPS), con Supabase cloud como base de datos. El widget está en JavaScript sin dependencias, con Shadow DOM, y se integra en cualquier web con una etiqueta script. 194 tests con pytest.

Stack: Python · FastAPI · PostgreSQL + pgvector (Supabase) · Ollama · API de Anthropic · Groq · Docker · Caddy · pytest · JavaScript

Asistente conversacional con IA integrado en app (prácticas DAM)

Marzo – septiembre 2026 · Equipo de 6 personas · Rol: backend, orquestación e infraestructura

- Diseñé los flujos en n8n y Dify que conectaban la app Flutter con los servicios de IA, incluido el contrato de datos entre la app y los flujos.
- Construí un agente acotado con system prompts y RAG para reducir las respuestas fuera de dominio.
- Integré Google Calendar vía OAuth para leer y crear eventos.
- Monté un servidor local compartido para que el equipo probara el mismo flujo, lo que unificó el entorno y liberó RAM de los portátiles.
- Definí el manejo de errores de los flujos y trabajé con Git en equipo (ramas y merges).

Stack: n8n · Dify · Groq · Flutter · Google Calendar OAuth · Docker

Sistema bancario: app cliente, panel de administración y chatbot con IA (proyecto personal)

2026 · [Repo](#)

- Dos apps Android nativas (Kotlin y Jetpack Compose) sobre una misma base de datos, con aislamiento de privilegios entre el panel de administración y la app de cliente.
- Base de datos en Supabase con Row Level Security y roles sobre datos sensibles. Transferencias entre usuarios con historial.
- Chatbot financiero que responde en lenguaje natural sobre los datos del propio usuario. Lo ejecuta un agente en n8n expuesto por webhook, desacoplado de la app.

Stack: Kotlin · Jetpack Compose · Supabase (PostgreSQL, RLS) · n8n · Groq · Ktor

Bot de Telegram para agendar citas por voz (proyecto personal)

2026 · [Repo](#)

- Pipeline de audio a cita: Make.com recibe el audio y Dify lo transcribe y extrae título, fecha y hora con system prompts.
- Crea el evento en Google Calendar vía OAuth y confirma al usuario. Los flujos se exportan sin credenciales y se pueden reimportar.

Stack: Telegram Bot API · Make.com · Dify · Gemini · Google Calendar API

Flag Challenge: juego web de banderas (proyecto personal, en producción)

2026 · [Repo](#) · [Demo](#)

- SPA en React y Vite desplegada en Vercel, con tres modos de juego y puntuación por velocidad.
- Backend en Supabase: perfiles, leaderboard en tiempo real con Realtime y políticas RLS que incluyen el modo invitado.

Stack: React · Vite · Supabase (Auth, RLS, Realtime) · Vercel

App de fitness y nutrición (proyecto personal, en desarrollo)

2026 · [Repo](#)

- App en Flutter con Clean Architecture y Riverpod, sobre Supabase con RLS. Está prevista la estimación de calorías a partir de fotos con IA.

Stack: Flutter · Dart · Riverpod · Supabase

EXPERIENCIA LABORAL

Operador de soporte técnico, Securitas Direct

Junio 2024 – septiembre 2025

- Diagnóstico remoto de averías, telemantenimiento de sistemas y resolución de incidencias técnicas.

Croupier, Casino de Barcelona (Grup Peralada)

Marzo 2022 – marzo 2023

- Gestión exacta de pagos y cobros y atención al cliente bajo presión.

FORMACIÓN

CFGS Desarrollo de Aplicaciones Multiplataforma (DAM), Davante MEDAC · en curso (2.º año)

Bachillerato Tecnológico, IES de Sales · 2016 – 2020

HABILIDADES TÉCNICAS

Backend: Python, FastAPI, SQL, PostgreSQL, pgvector, Supabase (RLS, Auth, Realtime), MongoDB, pytest.

IA aplicada: RAG, embeddings, evaluación de sistemas con LLM, agentes con herramientas, system prompts, API de Anthropic, Groq, Ollama.

Automatización: n8n, Dify, Make.com, webhooks, OAuth.

Infraestructura: Linux (Ubuntu), Docker y Docker Compose, Caddy, Cloudflare Workers, Vercel, Git y GitHub.

Otros lenguajes y móvil: Kotlin (Jetpack Compose), Dart (Flutter, Riverpod), JavaScript, TypeScript, React, C#.

Desarrollo asistido: Claude Code con especificaciones y skills.

IDIOMAS

Castellano y catalán (nativos) · Inglés (nivel alto)